

AutoCluster-OMP: A Lightweight Source-to-Source OpenMP Auto-Parallelizer for K-Means and Fuzzy C-Means Clustering

Mohamed Yasser Goma, Abdullah Ismail, Mohamed Khaled, Mohamed Saleh, Ahmed Sayed
School of Information Technology and Computer Science
Nile University, Egypt
{M.yasser2223, A.yasser2208, M.Khaled2241, M.Mahmoud2219, A.sayed2125}@nu.edu.eg

Abstract—Clustering algorithms such as K-Means and Fuzzy C-Means are widely used in data science and machine learning, but their repeated distance computations become expensive as the number of points, clusters, features, and iterations increases. Manual OpenMP parallelization can reduce runtime, but it requires the programmer to identify independent loops, avoid race conditions, select suitable schedules, and validate correctness. This paper presents AutoCluster-OMP, a lightweight source-to-source automatic parallelization prototype for clustering workloads. The tool analyzes sequential C implementations of K-Means and Fuzzy C-Means, detects clustering-specific loop patterns, classifies safe and unsafe loops, generates OpenMP directives, compiles sequential/manual/auto-generated versions, and evaluates performance using runtime, speedup, efficiency, and checksum-based correctness validation. The prototype targets the K-Means assignment loop and the Fuzzy C-Means membership-update loop, while warning about shared centroid-update patterns that may introduce races. Experimental results show that auto-generated OpenMP code achieves meaningful speedup while preserving correctness. On the Covertypes K-Means workload, AutoCluster-OMP achieved a $4.37\times$ speedup at 8 threads. For Fuzzy C-Means, the auto-generated version achieved up to $5.12\times$ speedup in the schedule and hyperparameter sweep.

Index Terms—Automatic parallelization, OpenMP, K-Means, Fuzzy C-Means, source-to-source transformation, clustering, parallel computing.

I. INTRODUCTION

Modern processors provide multiple cores, but sequential machine learning and data mining programs often fail to utilize this hardware efficiently. Clustering algorithms are a clear example. K-Means and Fuzzy C-Means repeatedly compute distances between data points and cluster centers, and their runtime grows with the number of points, clusters, features, and iterations.

K-Means is a hard clustering algorithm, where each point is assigned to exactly one cluster. Fuzzy C-Means is a soft clustering algorithm, where each point has a membership degree for every cluster. Although the algorithms differ in their assignment semantics, both contain dominant data-parallel loops. In K-Means, the point-assignment loop can process different

points independently. In Fuzzy C-Means, the membership-update loop can update membership rows independently.

OpenMP is a practical shared-memory programming model for multicore CPUs. However, inserting OpenMP directives manually requires experience. Some loops are safe to parallelize, while other loops, such as centroid-update loops, may contain shared writes and can introduce race conditions. Therefore, an automatic parallelization tool should not simply add OpenMP directives blindly. It should detect loop patterns, classify safety, and generate code only when the transformation is valid.

This project proposes AutoCluster-OMP, a lightweight source-to-source automatic parallelization prototype. The tool reads sequential C clustering code, detects supported loop patterns, generates OpenMP directives, compiles the generated code, benchmarks it against sequential and manual OpenMP baselines, and validates correctness using checksums.

The main contributions of this work are:

- A lightweight source-to-source automatic parallelization prototype for clustering workloads.
- Automatic detection of K-Means assignment loops and Fuzzy C-Means membership-update loops.
- Loop-safety classification that distinguishes independent loops from unsafe centroid-update loops.
- OpenMP code generation with configurable scheduling policies.
- Experimental evaluation using runtime, speedup, efficiency, schedule comparison, and checksum-based correctness validation.

II. RELATED WORK

A. Parallel K-Means

The parallelization of K-Means clustering is a central problem in high-performance data mining, focusing on partitioning the workload between distance calculations and centroid updates. While the assignment phase is embarrassingly parallel, the centroid update phase remains a primary bottleneck due to write contention caused by irregular scatter-style aggregations [1]. Recent innovations, such as Flash-KMeans, introduce

contention-free implementations using sort-inverse updates to transform high-contention atomic scatters into high-throughput reductions on modern GPUs [2]. However, achieving these gains on shared-memory architectures typically requires manual pragma insertion or the use of thread-local buffers, as automated tools often fail to manage these memory dependencies [1], [2].

The gap is that current research predominantly focuses on manual parallelization strategies that require significant programmer intervention to manage data dependencies and synchronization issues in centroid updates.

B. Parallel Fuzzy C-Means

Parallel Fuzzy C-Means introduces significantly higher computational complexity than hard clustering because each point has a membership value for each cluster. For large-scale data, research has turned to distributed frameworks such as Apache Spark, using distributed data abstractions to handle membership matrix updates across clusters [3]. Hybrid approaches such as FCMCAA utilize the Hadoop MapReduce framework to balance big data anonymization with parallel execution speed [4]. Despite these advancements for cloud-based environments, non-expert developers lack lightweight automation for local multicore systems, where the overhead of distributed clusters may outweigh the benefit of parallelization.

The gap is that there is a lack of accessible, lightweight automation tools that can recognize specific fuzzy clustering patterns on local shared-memory architectures without resorting to heavyweight distributed frameworks.

C. Automatic Parallelization

Automatic parallelization aims to transform sequential code into parallel code without human intervention. The Cetus Compiler v2.0 serves as a source-to-source infrastructure using compiler analysis and transformation techniques to insert OpenMP directives [5]. To improve optimization coverage, recent research such as the Parallel-Semantics Program Dependence Graph utilizes advanced compiler foundations to capture parallel program dependencies that traditional models may miss [6]. Furthermore, recent systems such as LLMA4OpenMP use large language models to assist automatic OpenMP parallelization [7]. While these tools advance the field, they may not provide domain-specific and explainable diagnostic feedback regarding why certain irregular clustering loops are unsafe for transformation.

The gap is that general-purpose automatic parallelization tools often operate as broad compiler frameworks or black-box systems. They do not directly target clustering-specific patterns or provide simple diagnostic explanations for assignment loops, membership-update loops, centroid-update hazards, and workload-aware OpenMP insertion.

III. PROPOSED SOLUTION

A. System Overview

AutoCluster-OMP is designed as a lightweight source-to-source transformation pipeline. The input is a sequential

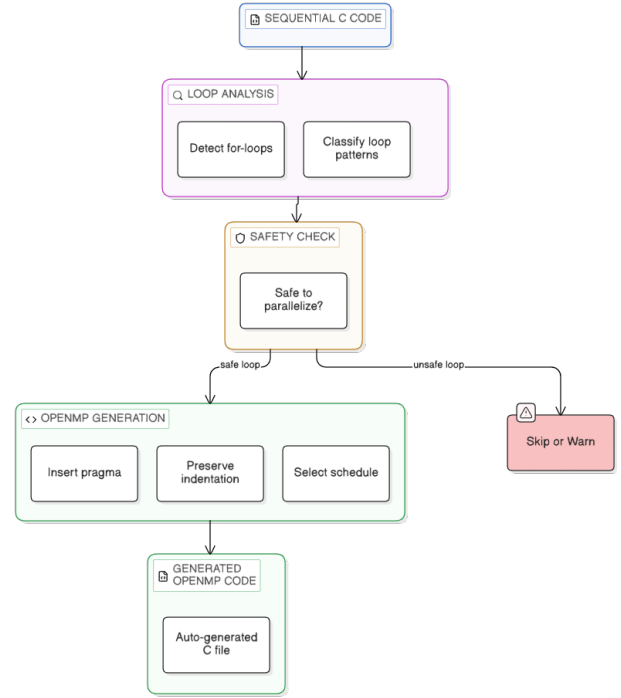


Fig. 1. AutoCluster-OMP source-to-source automatic parallelization pipeline.

C implementation of a supported clustering algorithm. The output is a generated C file containing OpenMP directives for safe loops.

The pipeline performs the following steps: read sequential C clustering code, detect C `for` loops, classify loop patterns using clustering-specific rules, check safety and workload size, insert OpenMP directives before safe loops, compile sequential/manual/auto-generated versions, benchmark performance, and validate correctness using checksums.

B. Supported Patterns

For K-Means, the target loop writes one cluster label per point:

Listing 1. K-Means target assignment loop.

```

for (int i = 0; i < n; i++) {
    ...
    labels[i] = best_cluster;
}
  
```

The generated loop is:

Listing 2. Generated OpenMP K-Means assignment loop.

```

#pragma omp parallel for schedule(static)
for (int i = 0; i < n; i++) {
    ...
    labels[i] = best_cluster;
}
  
```

This transformation is safe because each iteration writes to a unique `labels[i]` location, and centroids are read-only during the assignment phase.

TABLE I
LOOP CLASSIFICATION AND PARALLELIZATION DECISION

Classification	Decision	Reason
kmeans_assignment_loop	Parallelize	Independent writes to labels[i].
fuzzy_membership_loop	Parallelize	Independent membership-row updates.
centroid_shared_write	Warn	Possible shared writes to centroid accumulators.
possible_reduction_loop	Warn	Requires reduction or thread-local buffers.
unknown_loop	Skip	Unsupported pattern.

For Fuzzy C-Means, the target loop updates membership values:

Listing 3. Fuzzy C-Means target membership-update loop.

```
for (int i = 0; i < n; i++) {
  for (int c = 0; c < k; c++) {
    ...
    membership[i * k + c] = value;
  }
}
```

The outer loop is safe to parallelize because each point owns a distinct row in the membership matrix.

C. Loop Classification

AutoCluster-OMP classifies detected loops into five categories. Table I summarizes the classification rules.

Centroid-update loops require special handling because multiple threads may update the same cluster accumulator:

Listing 4. Unsafe shared centroid update pattern.

```
centroid_counts[cluster]++;
centroid_sums[cluster * d + f] += points[i * d + f];
```

Therefore, the prototype does not blindly parallelize these loops. It either warns about the race-condition risk or relies on manual/thread-local buffer versions for comparison.

D. Workload-Aware Decision

Small workloads may not benefit from OpenMP due to thread creation and scheduling overhead. Therefore, AutoCluster-OMP estimates computational work before parallelization. For K-Means, the estimated work is:

$$W_{KM} = N \times K \times D \times I, \quad (1)$$

where N is the number of points, K is the number of clusters, D is the number of features, and I is the number of iterations. For Fuzzy C-Means, the membership-update step performs additional cluster-to-cluster comparisons, so the estimated work is approximated as:

$$W_{FCM} = N \times K^2 \times D \times I. \quad (2)$$

TABLE II
AUTOCLUSTER-OMP IMPLEMENTATION MODULES

Module	Purpose
analyzer	Detects and classifies C loops.
generator	Inserts OpenMP pragmas.
compiler	Builds GCC commands and compiles C files.
benchmark	Runs experiments and calculates metrics.
reports	Exports CSV, JSON, and Markdown reports.
plotting	Generates runtime, speedup, and efficiency figures.
c_kernels	Contains K-Means and FCM C implementations.

IV. IMPLEMENTATION

The prototype uses Python for analysis, transformation, benchmarking, and reporting, and C for the clustering kernels. The main package modules are shown in Table II.

Each algorithm has three versions:

- Sequential C implementation.
- Manual OpenMP implementation.
- Auto-generated OpenMP implementation.

This design allows direct comparison between the baseline, human-written OpenMP, and tool-generated OpenMP.

V. EVALUATION AND RESULTS

A. Experimental Setup

The evaluation compares sequential, manual OpenMP, and auto-generated OpenMP implementations. The experiments use synthetic Gaussian datasets and the Coverttype real-world dataset. The Gaussian workload names were renamed in the paper for readability: paper_2d_100000 is reported as G2D-100K, and paper_3d_1000000 is reported as G3D-1M.

All C implementations were compiled with GCC using `-O2`.

OpenMP versions were compiled with `-fopenmp`. The evaluated thread counts were 1, 2, 4, and 8. Correctness was validated by comparing checksums against the sequential reference.

B. Metrics

Runtime is measured in seconds. Speedup is calculated as:

$$S = \frac{T_{seq}}{T_{par}}, \quad (3)$$

where T_{seq} is the sequential runtime and T_{par} is the parallel runtime. Efficiency is calculated as:

$$E = \frac{S}{P}, \quad (4)$$

where P is the number of OpenMP threads.

TABLE III
K-MEANS AUTO-GENERATED OPENMP RESULTS ON GAUSSIAN
DATASETS AT 8 THREADS

Workload	K	D	Time (s)	Speedup
G2D-100K	8	2	0.026	3.04×
G2D-200K	8	2	0.047	3.53×
G2D-500K	8	2	0.107	3.67×
G3D-100K	4	3	0.023	2.70×
G3D-200K	4	3	0.037	2.68×
G3D-400K	4	3	0.075	3.48×
G3D-800K	4	3	0.147	3.55×
G3D-1M	4	3	0.183	3.37×

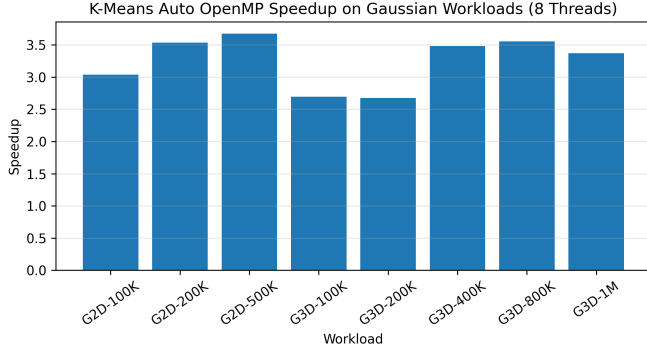


Fig. 2. Auto-generated K-Means OpenMP speedup across Gaussian workloads at 8 threads.

C. K-Means Synthetic Gaussian Results

Table III shows the K-Means auto-generated OpenMP results on Gaussian datasets at 8 threads. The generated OpenMP implementation achieved speedups between $2.68\times$ and $3.67\times$, while preserving correctness.

The best synthetic K-Means result was obtained on G2D-500K, where runtime decreased to 0.107 seconds and speedup reached $3.67\times$. The auto-generated implementation was also close to manual OpenMP performance. On G2D-500K, auto OpenMP achieved $3.67\times$, while manual OpenMP achieved $3.64\times$.

D. K-Means Real-World Dataset Result

Table IV shows the K-Means result on the Covertypes dataset, using 54 features, 7 clusters, and 30 iterations. On Covertypes, the auto-generated OpenMP version reduced runtime from 5.732 seconds to 1.311 seconds at 8 threads, achieving a $4.37\times$ speedup. All manual and auto OpenMP runs passed checksum validation.

E. Fuzzy C-Means Results

Fuzzy C-Means is more computationally expensive than K-Means because each point maintains a membership value for every cluster. Table V shows the results on a 10K-point, 15-cluster, 20-feature Gaussian workload with 30 iterations and fuzziness $m = 2.0$.

The auto-generated FCM OpenMP version reduced runtime from 20.952 seconds to 4.847 seconds at 8 threads,

TABLE IV
K-MEANS RESULTS ON THE COVERTYPE DATASET

Version	Threads	Time (s)	Speedup	Eff.
Sequential	1	5.732	1.00×	1.000
Manual OMP	2	3.260	1.76×	0.879
Manual OMP	4	1.936	2.96×	0.740
Manual OMP	8	1.407	4.07×	0.509
Auto OMP	2	3.196	1.79×	0.897
Auto OMP	4	1.949	2.94×	0.735
Auto OMP	8	1.311	4.37×	0.547

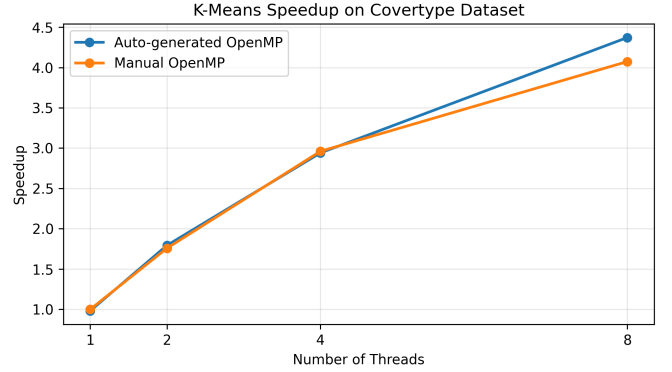


Fig. 3. K-Means speedup on the Covertypes dataset using manual and auto-generated OpenMP implementations.

achieving a $4.32\times$ speedup. This result is close to the manual OpenMP implementation, which achieved $4.19\times$ at 8 threads. Therefore, the generated OpenMP code can match manual parallelization for the supported FCM membership-update pattern.

F. Fuzzy C-Means Schedule and Hyperparameter Sweep

A larger FCM sweep was conducted using G2D-100K, G2D-200K, and G2D-500K datasets, 8 clusters, iteration counts of 5, 15, and 30, fuzziness values of 1.5, 2.0, and 2.5, and six OpenMP schedules: static, dynamic with chunks 64 and 256, guided with chunks 64 and 256, and auto. The sweep produced 648 auto-generated OpenMP FCM runs and 648 manual OpenMP runs. All OpenMP runs passed checksum validation.

Table VI summarizes the 8-thread auto-generated OpenMP FCM results by schedule.

The lowest median runtime was achieved by dynamic scheduling with chunk size 256. The highest observed speedup was $5.12\times$ using dynamic scheduling with chunk size 64 on the FCM-G2D-500K-k8-it30-m2.5 workload.

For a representative FCM configuration with fuzziness $m = 2.0$ and 30 iterations, the best auto-generated OpenMP results were:

- G2D-100K: 0.905 s, $3.95\times$ speedup using dynamic_256.
- G2D-200K: 1.890 s, $3.68\times$ speedup using guided_64.
- G2D-500K: 4.666 s, $3.73\times$ speedup using dynamic_256.

TABLE V
FUZZY C-MEANS MANUAL AND AUTO OPENMP RESULTS

Version	Threads	Time (s)	Speedup	Eff.
Sequential	1	20.952	1.00×	1.000
Manual OMP	2	11.200	1.87×	0.935
Manual OMP	4	6.427	3.26×	0.815
Manual OMP	8	4.997	4.19×	0.524
Auto OMP	2	11.454	1.83×	0.915
Auto OMP	4	6.482	3.23×	0.808
Auto OMP	8	4.847	4.32×	0.540

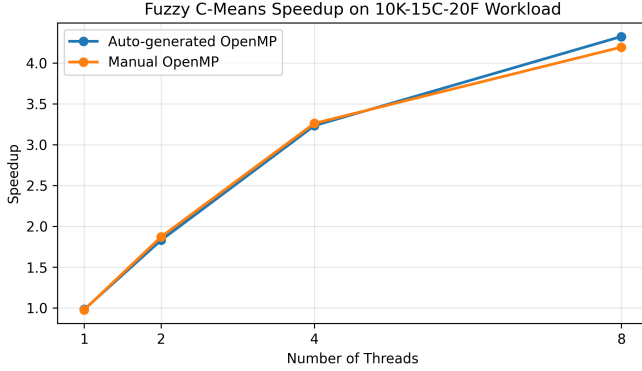


Fig. 4. Fuzzy C-Means speedup on the 10K-15C-20F workload using manual and auto-generated OpenMP implementations.

G. Discussion

The results confirm that AutoCluster-OMP can generate correct and efficient OpenMP code for both hard and soft clustering workloads. K-Means achieved strong speedup on both synthetic Gaussian workloads and the real-world Covertypes dataset. Fuzzy C-Means achieved higher absolute runtimes but also strong speedups because its membership-update loop performs significantly more computation.

The one-thread OpenMP versions are sometimes slower than the sequential baselines. This is expected because OpenMP introduces runtime overhead even when only one thread is used. However, as the number of threads increases, the generated OpenMP versions show clear runtime reductions. Efficiency decreases at higher thread counts. This behavior is expected in shared-memory parallel programs because of

OpenMP overhead, memory bandwidth limitations, synchronization costs, and remaining sequential sections such as centroid updates and initialization. Nevertheless, the speedup results show that the dominant assignment and membership-update loops are suitable targets for automatic parallelization.

VI. CONCLUSION AND FUTURE WORK

This paper presented AutoCluster-OMP, a lightweight source-to-source OpenMP auto-parallelizer for clustering workloads. The prototype detects K-Means assignment loops and Fuzzy C-Means membership-update loops, generates OpenMP directives, benchmarks the generated code, and

TABLE VI
FCM AUTO OPENMP SCHEDULE SUMMARY AT 8 THREADS

Schedule	Runs	Med. Time	Med. Speedup	Max
dynamic_256	27	1.896	3.77×	4.48×
guided_64	27	1.920	3.65×	4.36×
guided_256	27	1.931	3.62×	4.32×
static	27	2.037	3.38×	4.32×
auto	27	2.179	3.47×	4.37×
dynamic_64	27	2.466	3.70×	5.12×

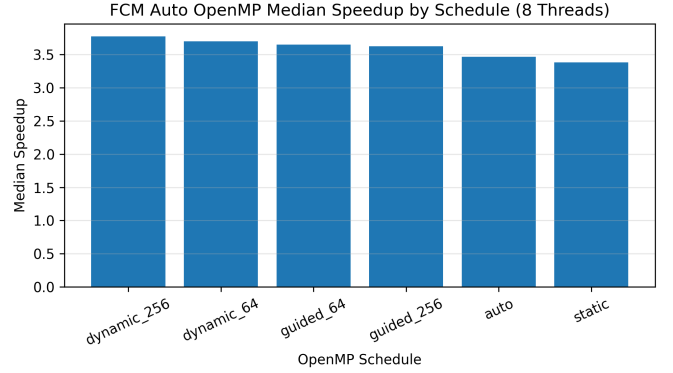


Fig. 5. Median speedup of auto-generated Fuzzy C-Means OpenMP code across scheduling policies at 8 threads.

validates correctness using checksums. Experimental results show that auto-generated OpenMP code can achieve meaningful speedup while preserving correctness. On Covertypes, auto-generated K-Means achieved 4.37×

 speedup at 8 threads. On Fuzzy C-Means, the auto-generated implementation achieved 4.32× speedup on the 10K-15C-20F workload and up to 5.12× in the schedule sweep.

Future work includes replacing the pattern-based parser with Clang AST analysis, improving automatic centroid-update transformations, adding automatic OpenMP schedule selection, supporting additional clustering algorithms, and extending the generator to OpenACC or CUDA for GPU execution.

REFERENCES

- [1] T. Raman and C. Columbus, "Parallel k-means clustering for customer segmentation in predictive marketing using openmp, mpi and cuda," in *2025 IEEE International Conference on Contemporary Computing and Communications*, 2025, metadata verified from public indexing pages.
- [2] S. Yang, H. Xi, Y. Zhao, M. Li, X. Fan, J. Zhang, H. Cai, Y. Lin, X. Li, K. Keutzer *et al.*, "Flash-kmeans: Fast and memory-efficient exact k-means," 2026. [Online]. Available: <https://arxiv.org/abs/2603.09229>
- [3] IEEE Access, "The parallel fuzzy c-median clustering algorithm using spark for the big data," *IEEE Access*, vol. 12, 2024, iEEE document 10684199.
- [4] J. U. Lawrance, J. V. N. Jesudhasan, and J. B. T. Rittammal, "Parallel fuzzy c-means clustering based big data anonymization using hadoop mapreduce," *Wireless Personal Communications*, vol. 135, no. 4, pp. 2077–2102, 2024.
- [5] A. Bhosale, R. Barua, R. Eigenmann *et al.*, "Automatic and interactive program parallelization using the cetus source to source compiler infrastructure v2.0," *Electronics*, vol. 11, no. 5, p. 809, 2022.

- [6] B. Homerding, A. Patel, E. A. Deiana, Y. Sui, Z. Tan, Z. Xu, B. R. Godala, D. I. August, and S. Campanoni, "The parallel-semantics program dependence graph," 2024. [Online]. Available: <https://arxiv.org/abs/2402.00986>
- [7] H. Zhu and W. Wang, "Llma4openmp: A large language model based agent for automatic openmp parallelization," in *2025 IEEE International Symposium on Parallel and Distributed Processing with Applications*, 2025, pp. 1201–1207.